

Faculdade de Engenharia da Universidade do Porto

# LCOM - CRIM

Autores:

|                 |             |
|-----------------|-------------|
| Rubem Neto      | [202207086] |
| Diogo Goiana    | [202207944] |
| Alexandre Silva | [202206633] |

Projeto final  
Laboratório de Computadores  
Turma 6 - Grupo 1



# Índice

|                                        |           |
|----------------------------------------|-----------|
| <b>1. Introdução</b>                   | <b>3</b>  |
| <b>2. Instruções de utilização</b>     | <b>4</b>  |
| 2.1 Ecrã inicial                       | 4         |
| 2.2 Escolha de nomes                   | 5         |
| 2.3 Conversa entre utilizadores        | 5         |
| 2.4 Novas mensagens                    | 6         |
| <b>3. Dispositivos</b>                 | <b>6</b>  |
| 3.1 I/O drivers usados                 | 6         |
| 3.2 Gráfica                            | 6         |
| 3.3 Teclado                            | 7         |
| 3.4 Rato                               | 7         |
| 3.5 RTC                                | 8         |
| 3.6 Porta série                        | 8         |
| <b>4. Estrutura</b>                    | <b>8</b>  |
| 4.1 graphic.c (35%)                    | 8         |
| 4.2 keyboard.c (20%)                   | 8         |
| 4.3 mouse.c (10%)                      | 9         |
| 4.4 rtc.c (5%)                         | 9         |
| 4.5 serial.c (10%)                     | 9         |
| 4.6 queue.c (5%)                       | 9         |
| 4.7 timer.c (5%)                       | 9         |
| 4.8 utils.c (3%)                       | 9         |
| 4.9 strings.c (5%)                     | 10        |
| 4.10 chat.c (1%)                       | 10        |
| 4.11 main.c (1%)                       | 10        |
| 4.12 Function call graph               | 10        |
| <b>5. Detalhes de implementação</b>    | <b>11</b> |
| 5.1 Troca de mensagens via Porta Série | 11        |
| <b>6. Conclusão</b>                    | <b>11</b> |

# 1. Introdução

Este relatório descreve o projeto final desenvolvido no âmbito da UC de Laboratório de Computadores, denominado de CRIM.

O objetivo principal deste projeto foi a criação de um chat que permite a comunicação entre dois utilizadores através da porta série, utilizando diversos dispositivos I/O.

O projeto foi desenvolvido com o intuito de aplicar os conhecimentos adquiridos ao longo do semestre, integrando vários conceitos de manipulação de hardware. A aplicação oferece uma interface intuitiva e funcionalidades robustas através de técnicas avançadas de programação em C.

## 2. Instruções de utilização

### 2.1 Ecrã inicial

A partir do ecrã inicial, o utilizador pode escolher entrar no chat ou sair. Sendo assim, o utilizador pode mexer o rato e seleccionar a sua opção com o botão esquerdo.



### 2.2 Escolha de nomes

Após a entrada no chat, o utilizador deve escolher o seu *username* para que possa ser identificado no envio de novas mensagens. Depois da escolha, deve esperar que o outro utilizador escolha o seu *username* para criar a ligação. Durante essa espera, o usuário já pode enviar mensagens para o correspondente.

DEFINE YOUR USERNAME

WAITING FOR OTHER USER

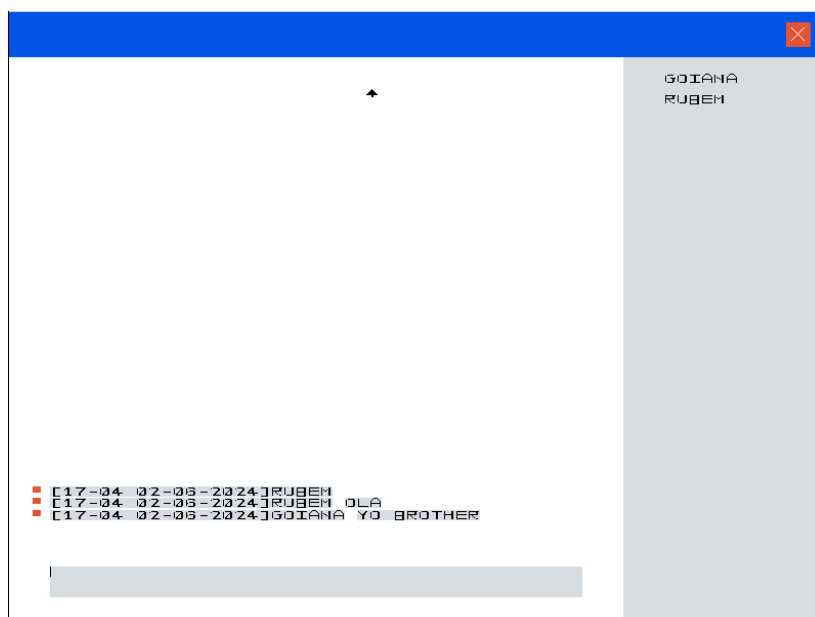
## 2.3 Conversa entre utilizadores

Depois da escolha de *usernames* para ambos os utilizadores, os dois podem enviar mensagens um para o outro. Para sair, apenas precisam de clicar no X no canto superior direito da tela ou clicar na tecla *ESC*.



## 2.4 Novas mensagens

Para cada nova mensagem, um quadrado ao lado da mensagem pisca para sinalizar que esta ainda não foi lida. Para o remover é necessário clicar com o botão direito no quadrado.



## 3. Dispositivos

### 3.1 I/O drivers usados

| Dispositivo | Uso                                                | Interrupções |
|-------------|----------------------------------------------------|--------------|
| Timer       | Controlo da frequência do programa. Blink feature. | Sim          |
| Teclado     | Escrita de mensagens.                              | Sim          |
| Rato        | Navegação na tela.                                 | Sim          |
| Gráfica     | Display das mensagens, menu e programa em geral.   | Não          |

|             |                                          |     |
|-------------|------------------------------------------|-----|
| Porta série | Comunicação P2P.                         | Sim |
| RTC         | Registo da data e hora de cada mensagem. | Não |

### 3.2 Gráfica

Para a implementação da gráfica optamos pelo modo **0x115**, que fornece um sistema de cor em **modo direto** com uma resolução de **800x600**.

Este modo de cor formata cada pixel com **24 bits totais, 8 para cada componente** ~ 24(8:8:8). O chat tira proveito de 7 cores no total, sendo elas:

- branco - 0xFFFFFF
- azul - 0x0156E6
- gray - 0xD8E0E5
- orange - 0xE25835
- azul claro - 0x89CFF0
- preto - 0x191970
- verde - 0x00FF00

Para uma melhor optimização do programa usamos a técnica de **double buffering via page flipping**, o que melhora substancialmente a rapidez e fluidez do chat.

Com o objetivo de mostrar o cursor do rato e as letras das teclas pressionadas, foi criada uma **fonte** composta por cada letra e número e também um **cursor** personalizado, através do formato **.xpm**.

A partir do driver do rato implementamos também **zonas de colisão** que permitem ao utilizador navegar pela tela, fechar o programa e dar uma mensagem como "lida".

Como forma de facilitar o desenvolvimento do chat, usamos certas funções da **VBE** para auxiliar o page flipping, a entrada e saída do modo de vídeo e também a leitura de ficheiros **.xpm**.

Algumas funções especiais utilizadas:

- *flip\_buffer()* e *clear\_buffer()*
- *update\_buffer()* e *update\_message\_buffer()*
- *draw\_messages()* e *draw\_mouse()*

### 3.3 Teclado

Este dispositivo é fulcral para o flow do nosso programa, sendo ele o principal responsável por **detectar o input de texto do utilizador** para este ser posteriormente convertido e traduzido no ecrã, pela gráfica.

Algumas funções especiais utilizadas:

- *retrieve\_letter()*

### 3.4 Rato

Como referido anteriormente, ao longo do desenvolvimento do programa foram criadas várias **zonas de colisão** com o objetivo de dar ao utilizador **liberdade de escolha na navegação**. Estas zonas são criadas em posições bem definidas da tela para detectar se o rato entra em contacto com alguma delas.

Cada colisão tem um objetivo específico e algumas só conseguem ser ativadas com um clique no botão esquerdo/direito, como por exemplo a opção de entrar/sair do chat ou a “leitura de uma mensagem” ao passar o rato por cima da mesma.

Algumas funções especiais utilizadas:

- `mouse_collision()`

### 3.5 RTC

Este dispositivo permite ao utilizador saber a **data e hora** de cada mensagem enviada/recebida através de uma leitura simples em **tempo real**. Para além disso, dependendo da hora do dia o chat alterna automaticamente entre **light/dark mode**.

Algumas funções especiais utilizadas:

- `bcd_to_bin()`
- `rtc_get_*`

### 3.6 Porta série

Este dispositivo é responsável pela comunicação entre diferentes computadores. Através da porta-série é possível, com o auxílio de uma estrutura de dados FIFO (`queue.c`), enviar um número arbitrário de mensagens para outro usuário. Para o envio de mensagens, usa-se o método de polling e para a receção de novas mensagens usa-se interrupções. Escolheu-se utilizar polling para enviar o momento em que vai ser enviada é conhecido. No entanto, o computador que recebe a mensagem, não sabe quando vai ser recebida, o que pode levar a um overhead de chamadas de polling.

Algumas funções especiais utilizadas:

- `ser_send_poll()`
- `ser_fifo_if()` (para interpretar as interrupções)

## 4. Estrutura

### 4.1 `graphic.c` (35%)

Este ficheiro é responsável por deter todas as funções que manipulam a tela e o seu estado. É este módulo que formata o ecrã, começa o programa em modo vídeo e gere o double buffering através de page flipping.

Principais estruturas de dados:

- array de apontadores para cada buffer
- struct com as informações do modo de vídeo

## **4.2 keyboard.c (20%)**

Este módulo contém todas as chamadas ao sistema precisas para manipular o dispositivo e as suas interrupções. As funções são responsáveis por ler o input do utilizador e verificar a validade deste, através de leituras e escritas em registos.

Para este dispositivo não foi necessário implementar estruturas de dados mais complexas de modo a melhorar a performance.

## **4.3 mouse.c (10%)**

Este ficheiro detém as funções necessárias para a movimentação do rato, também através de interrupções e chamadas ao sistema. Para além disso, é aqui que reside a lógica das colisões mencionada anteriormente e também a criação de cada packet.

Principais estruturas de dados:

- struct que define/cria um packet único
- struct que define uma zona de colisão

## **4.4 rtc.c (5%)**

Este módulo é responsável pela leitura da data/hora em tempo real para, posteriormente, cada mensagem poder ser melhor interpretada. Este está dividido em diversas funções, cada uma com uma finalidade distinta, mas que depois formam um resultado conjunto.

Principais estruturas de dados:

- struct que define uma data com todas as suas componentes

## **4.5 serial.c (10%)**

Este módulo caracteriza todo o trabalho pesado por trás da troca de mensagens. Suas funções designam-se por configurar os registos da porta-série até o envio de mensagens via polling e receção via interrupções. Ambas estas funcionalidades utilizam uma estrutura *FIFO* (First In First Out).

## **4.6 queue.c (5%)**

Este ficheiro contém a implementação de uma estrutura de dados First in First Out para auxiliar o funcionamento da porta série.

## **4.7 timer.c (5%)**

Este ficheiro contém todas as chamadas ao sistema para gerir as interrupções do dispositivo através de leituras e escritas em registos previamente definidos. Contém também a lógica que define a frequência da tela.

Para este dispositivo não foi necessário implementar estruturas de dados mais complexas de modo a melhorar a performance.

## 4.8 utils.c (3%)

Este módulo é constituído apenas por 3 funções que são usadas transversalmente pelos outros módulos todos. Estas facilitam a escrita e leitura de registos sem a necessidade de estruturas de dados mais complexas.

## 4.9 strings.c (5%)

Este ficheiro é simplesmente responsável por deter estruturas de dados e uma função que converte o input do utilizador (recebido no formato *scan\_code*) para o seu correspondente índice no alfabeto.

Principais estruturas de dados:

- dois arrays, um contém os *scan\_codes* e outro os caracteres

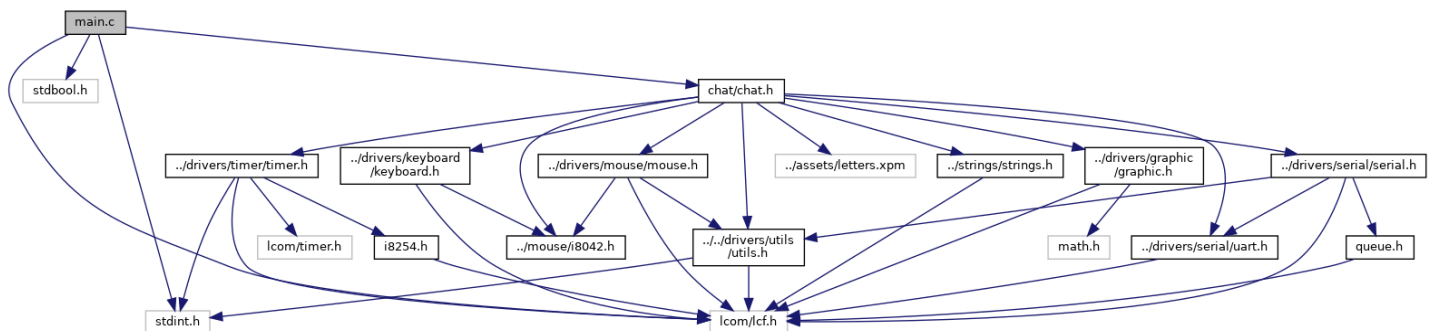
## 4.10 chat.c (1%)

Aqui reside a lógica primária do programa, responsável pela captação das diferentes interrupções e posterior chamada das principais funções. As estruturas de dados utilizadas neste ficheiro são comuns às anteriores, uma vez que este é o núcleo de todo o projeto.

## 4.11 main.c (1%)

Apenas o ficheiro “mãe” que inicia o programa.

## 4.12 Function call graph



Principais componentes associadas com o *driver\_receive()*:

- **Teclado:** Apanha e processa as interrupções do teclado. Converte cada *scan\_code* num inteiro para construir um array mapeado para letras/números. Envia a mensagem via porta série e atualiza o buffer das mensagens.
- **Porta série:** Apanha e processa as interrupções da porta série. Após receber e formatar uma mensagem completa, atualiza o buffer das mensagens.
- **Rato:** Apanha e processa as interrupções do rato. Responsável também por construir os packets necessários para a movimentação e colisões do dispositivo.

- **Timer:** Apanha e processa as interrupções do rato. Atualiza o buffer a cada tick e comanda imperativamente blinks na tela, quando necessários.

## 5. Detalhes de implementação

### 5.1 Troca de mensagens via Porta Série

Como descrito anteriormente, grande parte do projeto baseia-se na implementação e funcionamento da porta-série, portanto é importante salientar o funcionamento da mesma em maior detalhe. Primeiro, após o usuário enviar a mensagem (através da tecla Enter), parte da mensagem é transferida para um queue e enviada imediatamente para a outra porta série.

Quando o outro computador recebe parte da mensagem, transfere os dados recebidos para uma queue e depois é processada como parte de uma mensagem. No entanto, quando a mensagem acaba? Para resolver este problema, para sinalizar o fim da mensagem, envia-se um ponto final.

Portanto, o usuário envia a mensagem por partes através da FIFO e sinaliza o fim com um ponto final ("."), que permite ao outro usuário saber quando a mensagem está finalizada.

## 6. Conclusão

Após longas semanas de trabalho, o projeto foi implementado com todas as features pensadas inicialmente, assim como todos os dispositivos. O grupo de trabalho comprometeu-se a ir para além do obrigatório e desenvolveu componentes extra (e.g porta série) que tornaram o trabalho mais complexo.

Ao longo do desenvolvimento do projeto, o grupo debateu-se com algumas dificuldades na implementação dos dispositivos mais difíceis, porém tudo foi conseguido e estruturado da melhor forma.

A desistência bastante cedo de um dos membros do grupo trouxe alguma sobrecarga de trabalho para apenas 3 elementos que, muitas vezes, não foi bem dividida.

Olhando para o produto final ficam algumas coisas por adicionar, como a possibilidade de novos caracteres, assim como a introdução de letras minúsculas. Além disso, o design genérico do chat poderia ter sido aprimorado e refinado.

Para concluir, a realização deste projeto foi considerada bastante benéfica para os membros integrantes, uma vez que exigiu uma aprendizagem autónoma de conteúdos não lecionados.